

Credit Card Fraud Detection System Using Machine Learning

Introduction Credit card fraud is one of the most prevalent threats in online banking security, causing significant financial losses to both customers and institutions. Traditional systems are reactive — they only catch known fraud patterns and fail against new, evolving attack strategies. This project proposes a proactive ML-based system that detects fraudulent and anomalous transactions instantly, before the customer is even aware.

Problem Statement Given credit card transaction details, automatically detect whether a transaction is fraudulent or behaviorally anomalous — based on patterns such as unusual location, abnormal transaction amount, or sudden high-frequency payments — and return a confidence score indicating the fraud risk level.

Existing Methods & Current Gap

Existing Methods:

- **Rule-based systems** — Banks hardcode rules like "flag if amount exceeds threshold" or "flag if foreign transaction." Simple and fast, but rigid — attackers stay below the threshold deliberately.
- **Blacklists** — Known fraudulent cards, merchants, and IPs are blocked. Fails completely against new fraud not yet on the list.
- **Traditional ML** — Logistic Regression and basic Decision Trees trained on static datasets. Work reasonably but don't adapt to evolving fraud behavior and struggle with class imbalance.

Current Gap:

- Existing systems are **reactive** — fraud is caught only after it is reported or matches a known pattern
- Individual user behavior is **not modeled** — a ₹50,000 transaction may be normal for one customer and highly suspicious for another
- **Class imbalance** is poorly handled — models bias toward always predicting legitimate, missing actual fraud
- A **combined supervised + anomaly detection approach** is rarely implemented, leaving novel fraud patterns undetected

Dataset: [Kaggle Link](#)

The project uses a simulated credit card fraud dataset containing approximately 1.85 million transactions spanning January 2019 to December 2020. It captures behavior of 1,000 synthetic customers transacting with 800 merchants, including features such as

transaction amount, merchant category, geographical coordinates, and timestamp. The dataset reflects real-world class imbalance with fraudulent transactions forming a small minority.

Proposed Methodology

- **EDA & Preprocessing** — Explore data distributions, identify class imbalance, handle missing values and encode categorical features
- **Feature Engineering** — Derive behavioral features: time since last transaction, location change flag, amount deviation from customer average, transaction frequency within short time windows
- **Class Balancing via SMOTE** — Generate synthetic fraud samples to ensure the model learns fraud patterns effectively
- **Supervised Models** — Train and compare Logistic Regression (baseline), Random Forest, and XGBoost; evaluate using Precision, Recall, F1-Score, and AUC-ROC
- **Anomaly Detection Layer** — Isolation Forest operates unsupervised, flagging behavioral deviations from each customer's normal baseline — catching novel fraud that supervised models may miss
- **Combined Confidence Score** — Outputs from both layers are merged into a unified fraud risk score: Safe, Suspicious, or Fraud

System Architecture & Product: The trained model is deployed as a Flask-based web application with two functionalities:

- *Single Transaction Checker* — Enter transaction details manually, get an instant fraud prediction with confidence score
- *Batch Dashboard* — Upload a CSV of transactions and view a color-coded monitoring table simulating a bank analyst's interface

Expected Outcomes

- A trained and evaluated ML model with documented performance metrics across supervised and anomaly detection approaches
- A comparative analysis of Random Forest and XGBoost identifying the best performing model for credit card fraud detection
- A functional product — a web application where transaction details can be submitted and fraud risk is returned instantly, demonstrating practical deployment of the system